PYTHON

Module 11: Boolean Logic

The Binary Nature of Logic

In programming, you often need a simple 'Yes' or 'No' answer to direct the flow of your application. In Python, these are called Booleans, representing one of two values: `True` or `False`.

```
  Logic is the heartbeat of decision-making in code. Every time you compare two numbers or check a condition, Python evaluates that expression and replaces it with a Boolean value.
```

Comparison Evaluations

When you use comparison operators, Python returns a Boolean answer immediately.

- `10 > 9` evaluates to `True`.
- `10 == 9` evaluates to `False`.
- `10 < 9` evaluates to `False`.

This behavior allows you to create dynamic scripts that react differently based on the data they encounter.

The bool() Function: Truthiness

Python has a built-in bool() function that can evaluate any value and tell you its 'Truthiness'.

General Rule of Thumb:

- **Almost any value is True** if it has some sort of content.
- Any string is True (except empty strings).
- Any number is True (except 0).
- Any collection (list, tuple, dict) is True (except empty ones).

What Evaluates to False?

The list of values that evaluate to `False` is short and consistent. These are often referred to as 'Falsy' values:

- The keyword `False` itself.
- The value `None`.
- The number `0` (and `0.0`).
- Empty sequences: `""`, `()`, `[]`, `{}`.

Understanding these is critical for writing clean conditional statements (e.g., `if user_list:` will only run if the list is not empty).

Booleans in Functions

Functions can be designed specifically to return a Boolean value, serving as 'checkers' for your program logic.

Example:

python

```
def is_logged_in():
return True
```

You can also use built-in functions like `isinstance(variable, type)`, which returns `True` if the variable matches the specified type.

Conditional Flow Logic

Booleans are most powerful when used with `if` and `else` statements. This creates 'branches' in your code.

Workflow:

1. Python evaluates the condition (e.g., `age >= 18`).

2. If the result is `True`, the first block of code runs.

3. If the result is `False`, the `else` block runs.

This simple logic allows for the creation of complex software architectures.

Summary & Knowledge Check

At CodeHive, we use Booleans to keep code readable and efficient.

Exercise Question: What will be the result of `print(5 > 3)` ?

Answer: True

Key Takeaway: Booleans are capital-sensitive! Always use `True` and `False` with the first letter capitalized.